Escape the Geese – Architecture

Cohort 2 Team 3 – ‘Kostas Barmpis and Dimitris Kolovos’

Team members:

Sruthiksha Ananth

Louis Price

Sarah James

Ben Smejko

Henrik Bowden

Tim Giles

June Njiru

3.1 Introduction, languages and tools

This section presents the architecture of Escape the Geese and shows how it meets the requirements in Req1. Section 3.2 describes how the design evolved from our first CRC cards to the final version, section 3.3 gives the structural view (Figure 1), section 3.4 the behavioural view (Figures 2 and 3), and section 3.5 justifies the main design decisions and traces them to the requirements. Every interim version of our diagrams is on the Architecture page of our website.

Languages. We modelled the architecture in UML 2: class diagrams for structure, and sequence diagrams and a state machine for behaviour. UML is the standard notation for object-oriented design, so our own team and any team that continues the project in Assessment 2 can read it without explanation. We began with CRC (Class-Responsibility-Collaborator) cards, which are faster than UML to change while a design is still forming.

Table 3. Tools used for the architecture

Tool	Used for	Why we chose it	Alternative considered
PlantUML	Version 1 class and sequence diagrams; final sequence diagrams (Figures 2 and 3) and state machine	Diagrams are plain text, so they are stored in our GitHub repository, where every change is recorded	draw.io: easy to use, but files are hard to compare between versions
Mermaid in Lucidchart	Final class diagram (Figure 1)	Lucidchart generates the diagram from Mermaid code with right-angled connectors, and the team can view it online. Figure 1 is rendered from the same code as a vector image, so it stays sharp when zoomed	PlantUML: the 17-class diagram came out tangled and hard to read
Shared team document	CRC cards, version 1	Everyone could edit the cards during design discussions	Index cards: harder to share and keep as evidence
GitHub Pages	Full-size diagrams and every interim version	Free, built into our repository, and readable without logging in	Lucidchart share links, which may ask markers to sign in

3.2 Design process and evolution

Our architecture went through three versions, each driven by something we learned about the requirements or the framework. Every version is kept on our team website with its date, and the PlantUML sources in our repository record each change.

Stage 1: CRC cards (October 2026). We began with CRC cards drawn from the user requirements, which gave six classes: Game, Player, Goose, Item, Event and Score. Walking through gameplay scenarios showed two weaknesses. Game held eight responsibilities, from the countdown and pause state to score tracking and boundary checks, so almost any change would touch it. Some rules were still open: the Player card asked "Health bar ?", and Score ended the game once 100% of geese were dispersed.

Stage 2: first class and sequence diagrams (30 October 2026). We turned the cards into PlantUML class diagrams and six sequence diagrams. To settle the open question, this version gave Player a health value, and gave Goose a health value that items reduced. Rewriting the requirements from the full recording of our customer meeting then exposed three conflicts: the customer wanted a non-violent game with no health bar (CR_PEACE), a win that comes from clearing every wave rather than dispersing every goose (FR_WIN), and an in-game tutorial (FR_TUTORIAL). Starting implementation in LibGDX also showed that the framework already has a Game class and organises play into screens, so our own Game class clashed with it.

Stage 3: final design (12 November 2026). Version 2 resolves these problems with 17 classes in three layers (Figure 1). Game became Main and four LibGDX screens, and its remaining responsibilities moved into small classes that each own one job. Table 1 lists every change and the requirement behind it.

Table 1. Changes from version 1 to version 2

Version 1	Version 2	Why it changed	Requirements
Player and Goose had health; items reduced goose health	Player owns an ExhaustionMeter; geese are distracted, enraged or dispersed, never damaged	Customer asked for a non-violent game with no health bar	CR_PEACE, FR_EXHAUSTION_METER, FR_LOSE
One Game class ran the timer, pause, score, waves and win check	Main switches between MenuScreen, TutorialScreen, GameScreen and EndScreen; GameScreen runs one session	Game was too large and clashed with LibGDX's own Game class; screens keep menus separate from play	FR_TUTORIAL, FR_START_PAUSED, FR_PAUSE, CR_ENGINE
Game spawned waves and checked for the win	WaveManager spawns waves, runs the countdown and reports when all are cleared	The win became "clear every wave", so wave tracking needed one owner	FR_GEESE_WAVES, FR_WAVE_COUNTDOWN, FR_WIN

Version 1	Version 2	Why it changed	Requirements
Game counted down the 5 minutes	GameTimer, which stops while paused	The timer must stand still during a pause	FR_TIMER, FR_PAUSE
Player stored items itself	Player owns an Inventory	Separates item storage from movement; HUD reads it to draw icons	FR_START_ITEMS, FR_ITEM_ICON
One Item class with an effect type value	Abstract Item with Breadcrumbs and Pebble subclasses	Items differ in range and target, and a misused pebble enrages a goose	FR_ITEM_TYPES, FR_ITEM_MISUSE, QR_EXTENSIBILITY
One Event class	Abstract Event with ScoreBoost and GooseRush; trigger() always plays a sound and shows a visual cue	Every event needs two cues; a new event needs only a new subclass	FR_EVENT_AUDIO, FR_EVENT_VISUAL, QR_EXTENSIBILITY
Player and Score both held the score; Score ended the game at 100% dispersed	Score alone holds points and counters; WaveManager reports the win	Removes duplicate state; matches the new win condition	FR_SCORE_ITEMS, FR_USAGE_COUNTER, FR_WIN
No display class	HUD draws the timer, score, inventory, meter and wave number	All on-screen information in one place, readable in every display mode	FR_HUD_DISPLAY, QR_HUD_VISIBILITY

Looking ahead. Assessment 2 is likely to add difficulty levels (FR_DIFFICULTY), the rare escape win (FR_ESCAPE_WIN) and more items and events. Because items and events are subclasses of abstract classes and each game state is its own screen, most of these can be added as new classes rather than by changing existing ones (UR_EXTENSIBLE).

3.3 Structural view

Figure 1 shows the final UML 2 class diagram. Filled diamonds show composition (the part lives and dies with its owner), hollow diamonds aggregation, hollow triangles inheritance and dashed arrows dependencies. Each class is coloured by its layer, as set out in Table 2.

Table 2. Architectural layers

Layer	Classes	Responsibilities	Key requirements
Presentation	Main, MenuScreen, TutorialScreen, GameScreen, EndScreen, HUD, GameResult	Switching screens, input, the pause menu, on-screen information and the win or loss screen	FR_TUTORIAL, FR_START_PAUSED, FR_PAUSE_MENU, FR_HUD_DISPLAY, UR_GAME_END
Logic	Player, Inventory, ExhaustionMeter, Item, Breadcrumbs, Pebble, Goose, GooseState, WaveManager, Event, ScoreBoost, GooseRush, Score	The rules of the game: movement, items, goose behaviour, waves, events and scoring	UR_MOVEMENT, UR_ITEMS, UR_GEESE, UR_EVENTS, UR_SCORE
Support	GameMap, GameTimer, Settings	The campus map and its walls, the countdown, and volume and fullscreen options	FR_MAP_BOUNDS, FR_LANDMARKS, FR_TIMER, FR_VOLUME, FR_FULLSCREEN
Framework (LibGDX, not drawn)	Game, Screen, SpriteBatch, Vector2, Rectangle	The main loop, rendering, input and geometry our classes build on	CR_JAVA, CR_ENGINE

How the classes relate. Ownership organises the structure. Main owns Settings and switches between the four screens. GameScreen owns everything that exists for one play session (GameMap, Player, WaveManager, GameTimer, Score, HUD and at least five events), so a restart builds a fresh session and nothing is saved, as the customer asked. Player owns its Inventory and ExhaustionMeter, and WaveManager owns the geese of each wave. Items are aggregated rather than composed, because the map and inventory hold items they did not create. Dashed dependencies show use without ownership: geese fill the meter and avoid the walls in GameMap; items disperse, distract or enrage geese; events set the score multiplier, resize the next wave, spawn items and read the volume. Item and Event are abstract, so new items and events are added as subclasses without changing the classes that use them (QR_EXTENSIBILITY). Class names are identical in the CRC cards, every figure and the code.

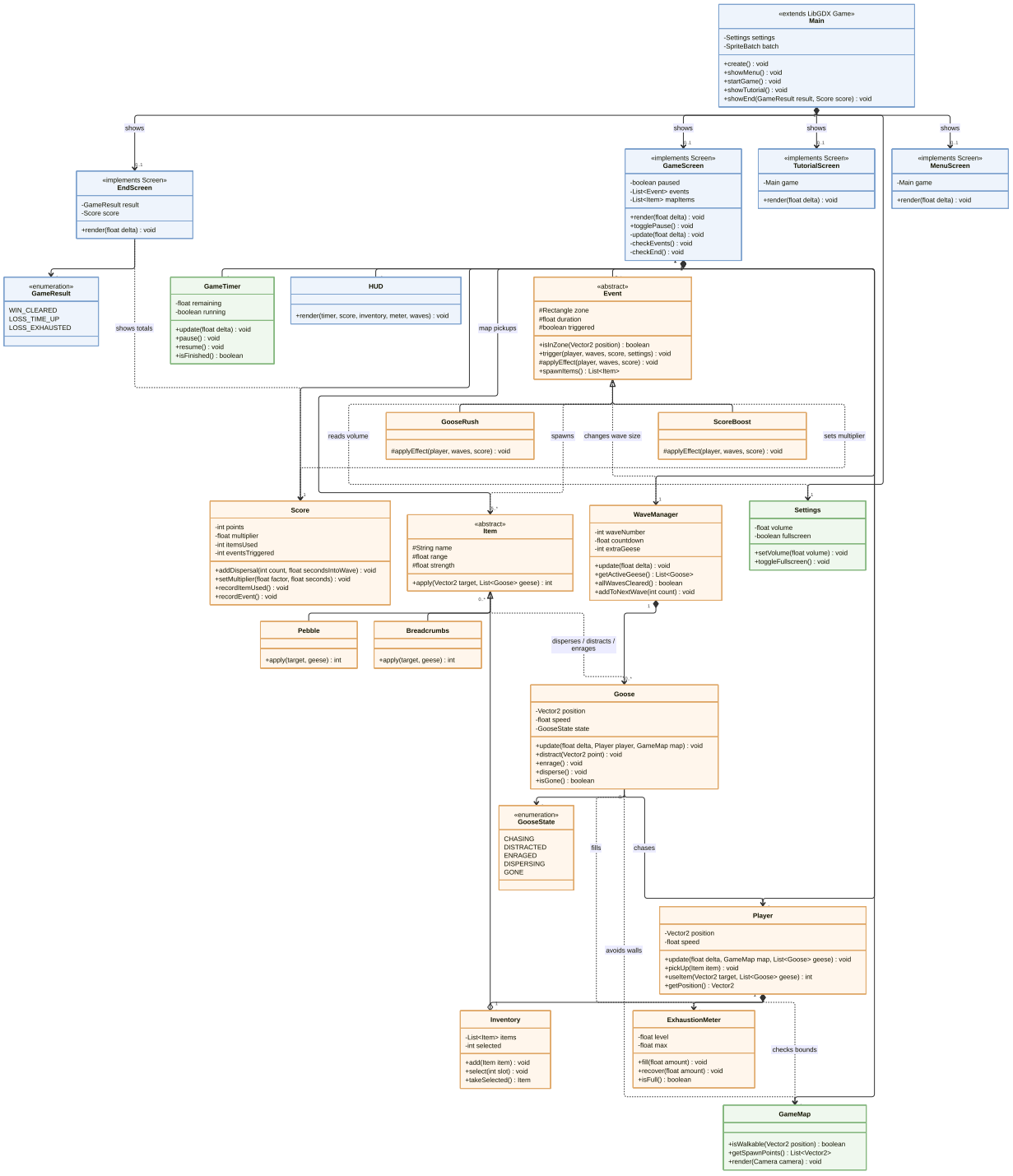


Figure 1. Final class diagram (version 2, 12 November 2026). Blue = presentation, orange = logic, green = support. HUD reads GameTimer, Score, Inventory, ExhaustionMeter and WaveManager to draw the screen; these links are omitted for clarity. Full-size, zoomable version: sruthiksha-source.github.io/Escape-the-Geese-Web/assets/arch/class-diagram-v2.pdf

3.4 Behavioural view

We used UML sequence diagrams for the two classes that coordinate the most behaviour: Player, through which every player action passes (Figure 2), and GameScreen, which runs each frame and decides the result (Figure 3). **loop**, **alt** and **opt** frames show repetition, alternatives and optional steps, and every message uses a method name from Figure 1. A state machine for Goose, the class whose behaviour changes most over time, is on our website.

3.4.1 Player uses an item and clears a wave

Requirements: FR_ITEM_DISPERSE, FR_ITEM_TYPES, FR_ITEM_MISUSE, FR_SCORE_ITEMS, FR_SCORE_SPEED, FR_USAGE_COUNTER, FR_GEESE_WAVES, FR_WAVE_COUNTDOWN

Classes involved: GameScreen, Player, Inventory, Item (Breadcrumbs, Pebble), Goose, Score, WaveManager

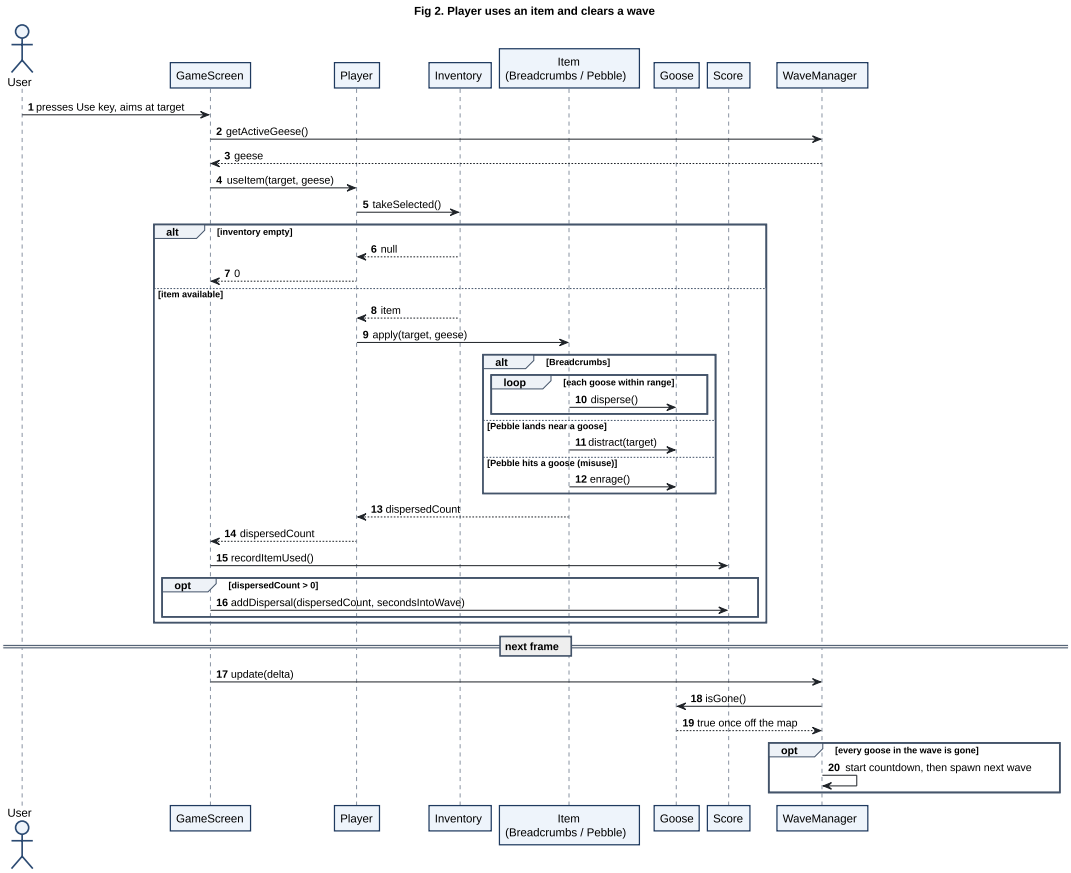


Figure 2. Sequence diagram: the player uses an item and clears a wave. Full size: team website (#seq-v2).

When the user presses the Use key, GameScreen passes the active geese to Player.useItem(). Player takes the selected item from its Inventory and calls apply() where it lands. Breadcrumbs disperse every goose in range, while a pebble distracts a goose nearby but enrages one it hits (FR_ITEM_MISUSE). Score records the use and awards points for each dispersal, more early in a wave (FR_SCORE_SPEED). On the next frame, WaveManager removes geese that have left the map and, once a wave is empty, counts down to the next (FR_WAVE_COUNTDOWN).

3.4.2 One GameScreen frame: events and the end of the game

Requirements: FR_START_PAUSED, FR_PAUSE, FR_TIMER, FR_EXHAUSTION_METER, FR_EVENT_AUDIO, FR_EVENT_VISUAL, FR_USAGE_COUNTER, FR_HUD_DISPLAY, FR_WIN, FR_LOSE

Classes involved: Main, GameScreen, GameTimer, Player, WaveManager, Goose, ExhaustionMeter, Event (ScoreBoost, GooseRush), Score, HUD

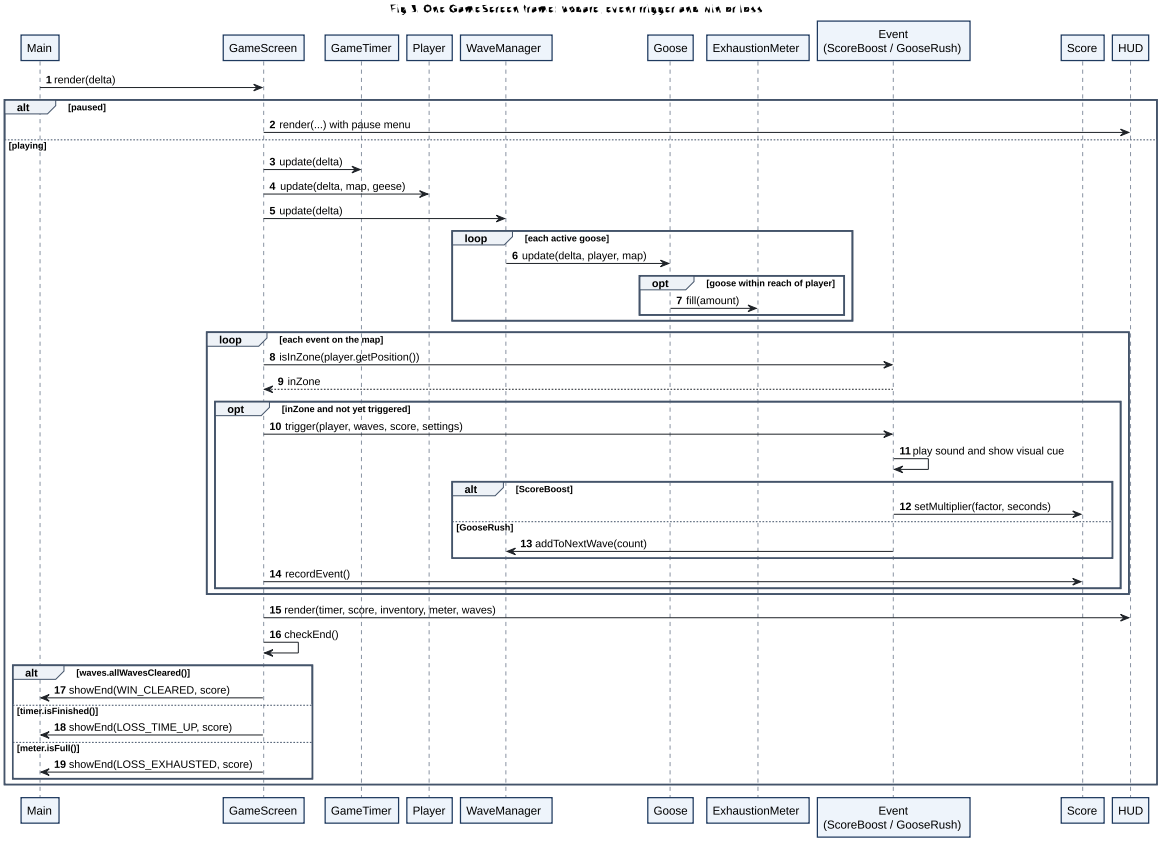


Figure 3. Sequence diagram: one frame of play, including event triggering and the win and loss checks. Full size: [team website (#seq-v2)](#).

While paused, GameScreen draws only the pause menu and the timer stands still (FR_PAUSE). Otherwise it updates the timer, player and geese, and any goose within reach fills the ExhaustionMeter. An event triggers once when the player enters its zone: trigger() always plays its sound and shows its visual cue, then the subclass applies its effect, a score multiplier (ScoreBoost) or a larger next wave (GooseRush). Finally GameScreen checks for a win (every wave cleared) or a loss (time up or meter full).

3.5 Justification and traceability

Each major design decision was compared with at least one alternative and is tied to the requirements it serves (Table 4).

Table 4. Design decisions

Decision	Alternative considered	Reason	Requirements
One LibGDX Screen class per game state	One class with a state flag (our version 1 Game class)	Each screen owns its own input and drawing, so adding or changing a screen does not affect the others	FR_TUTORIAL, FR_START_PAUSED, QR_EXTENSIBILITY
GameScreen owns one play session through composition	Global or static game state	Every object lives for exactly one game, so a restart starts clean and nothing is saved, as the customer asked	FR_WIN, FR_LOSE, out of scope: saving (Q15)
Abstract Item and Event classes with subclasses	One class with an effect-type value (our version 1)	A new item or event is a new subclass, with no type checks spread through Player and Goose	FR_ITEM_TYPES, FR_EVENTS_COUNT, QR_EXTENSIBILITY
Event.trigger() plays the sound and shows the visual cue, then calls applyEffect()	Each event handles its own sound and cue	Every event is guaranteed to give two cues, which a new subclass cannot forget	FR_EVENT_AUDIO, FR_EVENT_VISUAL
Goose behaviour as states (Chasing, Distracted, Enraged, Dispersing)	Health values on geese (our version 1)	Meets the non-violent brief, and makes pebble misuse a clear state change	CR_PEACE, FR_ITEM_MISUSE
ExhaustionMeter as its own class, owned by Player	A health field on Player	Keeps the loss rule in one place that GameScreen checks and HUD draws	FR_EXHAUSTION_METER, FR_LOSE

Traceability. The requirements lines above Figures 2 and 3 list the IDs each scenario covers, Table 2 maps each layer to its requirements, and Table 5 traces each user requirement forward to the classes and figures that satisfy it. Class names are identical in the CRC cards, diagrams, this report and the code.

Table 5. User requirements traced to the design

User requirement	Classes that satisfy it	Shown in
UR_MOVEMENT	Player, GameMap	Figures 1 and 3
UR_GEESE	Goose, GooseState, WaveManager, ExhaustionMeter	Figures 1 and 3; state machine on website
UR_ITEMS	Item, Breadcrumbs, Pebble, Inventory	Figures 1 and 2
UR_EVENTS	Event, ScoreBoost, GooseRush	Figures 1 and 3
UR_GAME_END	GameScreen, WaveManager, GameTimer, ExhaustionMeter, EndScreen	Figures 1 and 3
UR_SCORE	Score, HUD, EndScreen	Figures 1, 2 and 3
UR_TUTORIAL	TutorialScreen, MenuScreen	Figure 1
UR_ACCESSIBILITY	Settings (volume, fullscreen), GameScreen (pause), Event (sound and visual cue), HUD	Figures 1 and 3
UR_RELATE	GameMap (York landmarks)	Figure 1
UR_PERFORMANCE	Main (loads assets once), GameScreen (one update and draw per frame)	Figures 1 and 3
UR_EXTENSIBLE	Abstract Item and Event; one Screen per game state	Figure 1

Two May requirements are deliberately left for Assessment 2: difficulty levels (FR_DIFFICULTY) and the escape win (FR_ESCAPE_WIN). Both fit the current structure without redesign.